

Maximum Reachable Index Difference

Problem

Given a string `s` containing lowercase English alphabets.

Start from any index containing the character `'a'`.

In each jump operation:

- Move to any index on the right.
- The character at the destination must be the immediate next letter of the current character.

The character sequence is:

```
a → b → c → d → e → ...
```

Continue jumping until no further jump is possible.

Find the maximum possible difference between the starting index and the ending index.

If it is not possible to choose a starting index containing `'a'`, return `-1`.

Examples

Example 1

Input:

```
s = "aaabcb"
```

Index representation:

```
index:  0  1  2  3  4  5
char:   a  a  a  b  c  b
```

Starting from index `0`:

Possible jumps:

```
0 ('a') → 3 ('b') → 4 ('c')
```

Distance:

$$4 - 0 = 4$$

Another possible path:

$$0 \text{ ('a')} \rightarrow 5 \text{ ('b')}$$

Distance:

$$5 - 0 = 5$$

Therefore:

$$\text{Answer} = 5$$

Example 2

Input:

```
s = "xynjir"
```

There is no 'a'.

Therefore:

$$\text{Answer} = -1$$

Example 3

Input:

```
s = "abcbzzd"
```

Index representation:

index:	0	1	2	3	4	5	6
char:	a	b	c	b	z	z	d

Best path:

0 ('a') → 1 ('b') → 2 ('c') → 6 ('d')

Distance:

$6 - 0 = 6$

Therefore:

Answer = 6

1. Understand the Jump Rule

The next character is always the immediate next alphabet character.

a → b
b → c
c → d
d → e
...

If the current character is:

'a'

then the next required character is:

'b'

If the current character is:

```
'b'
```

then the next required character is:

```
'c'
```

We can calculate the next character using:

```
next_char = chr(ord(current_char) + 1)
```

Example:

```
current_char = 'a'  
  
ord('a') = 97  
  
97 + 1 = 98  
  
chr(98) = 'b'
```

Therefore:

```
next_char = chr(ord(current_char) + 1)
```

2. What Makes a Jump Valid?

Suppose we are currently at index **i**.

We can jump to index **j** only when:

```
j > i
```

because the destination must be on the right.

And:

```
s[j] == immediate next character after s[i]
```

Therefore, a valid jump is:

```
i → j
```

when:

```
j > i
```

and:

```
s[j] == chr(ord(s[i]) + 1)
```

For example:

```
s[i] = 'b'
```

Then:

```
next character = 'c'
```

So we can jump from **i** to any **j** satisfying:

```
j > i  
s[j] == 'c'
```

3. First Logic-Building Question

Ask:

What can be the starting index?

Only an index containing:

```
'a'
```

Therefore:

```
for every index i:  
    if s[i] == 'a':  
        try starting from i
```

In code:

```
for i in range(n):  
    if s[i] == 'a':  
        # Try starting from i
```

After starting from 'a', the character sequence must be:

```
a → b → c → d → ...
```

4. Important Observation: There Can Be Multiple Choices

Consider:

```
s = "aaabcb"  
  
index:  0  1  2  3  4  5  
char:   a  a  a  b  c  b
```

Start from:

```
index 0 = 'a'
```

The next required character is:

```
'b'
```

There are two possible 'b' positions:

```
index 3
index 5
```

Therefore:

```
0 → 3
```

and:

```
0 → 5
```

are both valid.

This is why we cannot simply choose the first 'b'.

We also cannot blindly choose the last 'b'.

For example:

```
s = "abcbzzd"

index:  0  1  2  3  4  5  6
char:   a  b  c  b  z  z  d
```

From index 0:

Possible 'b' positions:

```
index 1
index 3
```

If we choose:

```
0 → 3
```

then index 3 contains 'b'.

We need 'c' after index 3.

There is no 'c' after index 3.

So:

$$0 \rightarrow 3$$

gets stuck.

But if we choose:

$$0 \rightarrow 1$$

then:

$$1 ('b') \rightarrow 2 ('c') \rightarrow 6 ('d')$$

Therefore:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 6$$

Distance:

$$6 - 0 = 6$$

This proves:

Greedy choice of first next character is not enough.

Greedy choice of last next character is also not enough.

We need to consider all possible transitions.

This naturally suggests:

DFS / Recursion

5. Brute Force Logic

For every possible starting 'a':

1. Start at index i .
2. Calculate the required next character.
3. Search every index j on the right.
4. If $s[j]$ is the required next character:
try jumping to j .
5. Recursively continue from j .
6. Find the farthest ending index.
7. Calculate:
 $\text{ending_index} - \text{starting_index}$
8. Keep the maximum answer.

6. Graph Interpretation

Every index can be considered a node.

For:

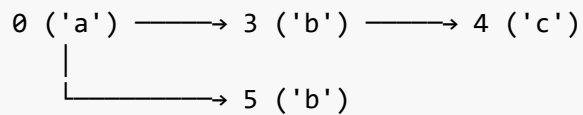
```
s = "aaabcb"
```

index:	0	1	2	3	4	5
char:	a	a	a	b	c	b

Some valid edges are:

```
0 → 3  
0 → 5  
1 → 3  
1 → 5  
2 → 3  
2 → 5  
3 → 4
```

The graph can be visualized as:



Every edge moves to the right:

$$i \rightarrow j$$

where:

$$j > i$$

Therefore, cycles are impossible.

This is a DAG-like structure.

7. Brute Force DFS State

Define:

$$\text{dfs}(i)$$

as:

The farthest index reachable starting from index i .

Suppose:

$$s[i] = 'b'$$

Then:

$$\text{next character} = 'c'$$

Search every index j satisfying:

```
j > i
```

and:

```
s[j] == 'c'
```

For every valid j :

```
i → j
```

Then we recursively calculate:

```
dfs(j)
```

Therefore:

```
dfs(i) = maximum result among all valid dfs(j)
```

If no valid jump exists:

```
dfs(i) = i
```

because we cannot move anywhere else.

8. Brute Force Recursive Code

```
def maxIndexDifference(s):  
    n = len(s)  
  
    def dfs(i):  
  
        current_char = s[i]  
  
        # Required next character  
        next_char = chr(ord(current_char) + 1)  
  
        # If no jump is possible,
```

```
# we can at least stay at i
best_end = i

# Try every position on the right
for j in range(i + 1, n):

    # Valid jump
    if s[j] == next_char:

        best_end = max(
            best_end,
            dfs(j)
        )

    return best_end

answer = -1

# Try every possible starting 'a'
for i in range(n):

    if s[i] == 'a':

        ending_index = dfs(i)

        answer = max(
            answer,
            ending_index - i
        )

return answer
```

9. Brute Force Dry Run

Consider:

```
s = "aaabcb"
```

Index representation:

```
index:  0  1  2  3  4  5
char:   a  a  a  b  c  b
```

Start from index 0.

We call:

```
dfs(0)
```

Current character:

```
s[0] = 'a'
```

Required next character:

```
'b'
```

Search to the right:

```
index 1 → 'a' → invalid  
index 2 → 'a' → invalid  
index 3 → 'b' → valid  
index 4 → 'c' → invalid  
index 5 → 'b' → valid
```

Therefore:

```
dfs(0)
```

has two possible branches:

```
0 → 3
```

and:

```
0 → 5
```

Branch 1: 0 → 3

At index 3:

```
s[3] = 'b'
```

Required next character:

```
'c'
```

Search after index 3:

```
index 4 → 'c' → valid
```

Therefore:

```
3 → 4
```

Now at index 4:

```
s[4] = 'c'
```

Required next character:

```
'd'
```

Search after index 4:

```
index 5 → 'b'
```

There is no 'd'.

Therefore:

```
dfs(4) = 4
```

So:

$$\text{dfs}(3) = 4$$

This path gives:

$$0 \rightarrow 3 \rightarrow 4$$

Ending index:

$$4$$

Distance:

$$4 - 0 = 4$$

Branch 2: $0 \rightarrow 5$

At index 5:

$$s[5] = 'b'$$

Required next character:

$$'c'$$

There is no index after 5.

Therefore:

$$\text{dfs}(5) = 5$$

This path gives:

$$0 \rightarrow 5$$

Ending index:

5

Distance:

$$5 - 0 = 5$$

Final Result

From index 0:

branch 1 gives ending index 4

branch 2 gives ending index 5

Therefore:

$$\text{dfs}(0) = \max(4, 5)$$

$$\text{dfs}(0) = 5$$

Distance:

$$5 - 0 = 5$$

Answer:

5

10. Complexity of Brute Force DFS

For every index, we may scan all indices to its right.

For example:

index 0 → scan 1 to n-1

```
index 1 → scan 2 to n-1  
index 2 → scan 3 to n-1  
...
```

This can be:

```
O(n2)
```

Additionally, without memoization, the same state can be calculated repeatedly.

For example:

```
a → b1 → c  
a → b2 → c
```

The result for the same `c` state may be calculated multiple times.

Therefore, the brute force solution is inefficient.

11. First Optimization: Memoization

Notice that:

```
dfs(i)
```

always represents the same state.

For a particular index `i`, the farthest reachable ending index is fixed.

Therefore, we can store:

```
memo[i] = farthest ending index reachable from i
```

Before calculating:

```
dfs(i)
```

check:

```
if memo[i] has already been calculated:  
    return memo[i]
```

12. Memoized DFS Code

```
def maxIndexDifference(s):  
    n = len(s)  
  
    memo = [-1] * n  
  
    def dfs(i):  
  
        if memo[i] != -1:  
            return memo[i]  
  
        current_char = s[i]  
  
        next_char = chr(ord(current_char) + 1)  
  
        best_end = i  
  
        for j in range(i + 1, n):  
  
            if s[j] == next_char:  
  
                best_end = max(  
                    best_end,  
                    dfs(j)  
                )  
  
        memo[i] = best_end  
  
        return best_end  
  
    answer = -1  
  
    for i in range(n):  
  
        if s[i] == 'a':  
  
            ending_index = dfs(i)  
  
            answer = max(  
                answer,  
                ending_index - i  
            )
```

```
return answer
```

13. Complexity of Memoized DFS

There are n possible states:

```
dfs(0)
dfs(1)
dfs(2)
...
dfs(n - 1)
```

Each state may scan the remaining part of the string.

Therefore:

```
Time Complexity =  $O(n^2)$ 
```

Memoization removes repeated recursive calculations, but each index can still scan many future indices.

So we need another optimization.

14. Search for the Real Bottleneck

The expensive part is:

```
for j in range(i + 1, n):
    if s[j] == next_char:
        use dfs(j)
```

For every i , we scan the entire right side to find the next character.

But the alphabet has only:

```
26 lowercase characters
```

This gives us an opportunity.

Instead of searching the entire right side repeatedly, can we store the best result for each character?

This leads to:

```
Right-to-left Dynamic Programming
```

15. Right-to-Left Thinking

Suppose we process the string from right to left.

Why is that useful?

When we are currently at index i , all indices to the right have already been processed.

Therefore, the answer for the required next character is already available.

Example:

```
current character = 'b'
```

Required next character:

```
'c'
```

If we already know:

```
best['c']
```

then we do not need to scan the entire right side again.

We can directly use:

```
best['c']
```

16. Define the DP Meaning

We need to carefully define:

```
best[c]
```

Meaning:

Among all occurrences of character `c` that have already been processed on the right side, what is the farthest ending index reachable from one of those occurrences?

For example:

```
best['c'] = 6
```

means:

There exists an occurrence of `'c'` on the right from which we can eventually reach index `6`.

Then if we are currently at:

```
s[i] = 'b'
```

we need:

```
'c'
```

So:

```
dp[i] = best['c']
```

If no `'c'` exists to the right:

```
dp[i] = i
```

because we cannot jump.

17. Why Process Right to Left?

Consider:

```
s = "abcbzzd"
```

```
index:  0  1  2  3  4  5  6  
char:   a  b  c  b  z  z  d
```

We want to calculate the answer for index 0.

At index 0:

```
'a'
```

We need:

```
'b'
```

There are multiple 'b' positions:

```
index 1  
index 3
```

We need to know which 'b' eventually gives the farthest result.

If we process from right to left:

```
index 6  
index 5  
index 4  
index 3  
index 2  
index 1  
index 0
```

then when we reach index 0, all possible future states have already been solved.

18. Complete Right-to-Left Dry Run

Input:

```
s = "abcbzzd"
```

Index representation:

```
index:  0  1  2  3  4  5  6  
char:   a  b  c  b  z  z  d
```

We maintain:

```
best[character]
```

Initially:

```
best = empty
```

Step 1: i = 6

Current:

```
s[6] = 'd'
```

Required next character:

```
'e'
```

There is no 'e' processed on the right.

Therefore:

```
farthest = 6
```

Store:

```
best['d'] = 6
```

Meaning:

Starting from this 'd', the farthest reachable index is 6.

Step 2: i = 5

Current:

```
s[5] = 'z'
```

There is no next lowercase character after 'z'.

Therefore:

```
farthest = 5
```

Store:

```
best['z'] = 5
```

Step 3: i = 4

Current:

```
s[4] = 'z'
```

Again, no next lowercase character exists.

Therefore:

```
farthest = 4
```

But there is already:

```
best['z'] = 5
```

We keep the better result:

```
best['z'] = max(5, 4)
```

```
best['z'] = 5
```

Step 4: i = 3

Current:

```
s[3] = 'b'
```

Required next character:

```
'c'
```

Do we currently know a reachable 'c' on the right?

No.

Therefore:

```
farthest = 3
```

Store:

```
best['b'] = 3
```

This means:

From index 3, we cannot reach a 'c' after it, so we stop at index 3.

Step 5: i = 2

Current:

```
s[2] = 'c'
```

Required next character:

```
'd'
```

We already know:

```
best['d'] = 6
```

Therefore:

```
farthest = 6
```

Store:

```
best['c'] = 6
```

Meaning:

```
index 2 ('c') → index 6 ('d')
```

So:

```
best['c'] = 6
```

Step 6: i = 1

Current:

```
s[1] = 'b'
```

Required next character:

```
'c'
```

We already know:

```
best['c'] = 6
```

Therefore:

```
farthest = 6
```

So:

```
best['b'] = max(  
    existing best['b'],  
    6  
)  
  
best['b'] = max(3, 6)  
  
best['b'] = 6
```

This is important.

There are two 'b' positions:

```
index 3 → farthest ending index 3  
  
index 1 → farthest ending index 6
```

Therefore, the best 'b' state is:

```
best['b'] = 6
```

Step 7: i = 0

Current:

```
s[0] = 'a'
```

Required next character:

```
'b'
```

We already know:

```
best['b'] = 6
```

Therefore:

```
farthest = 6
```

Distance:

```
6 - 0 = 6
```

Answer:

```
6
```

19. The Main Optimization

The brute-force approach does:

```
For every i:  
    scan all j > i  
    find the required next character
```

This costs:

```
 $O(n^2)$ 
```

The optimized approach does:

```
Process from right to left.  
  
For each index:
```

```
calculate the required next character
look up best[next_character]
update best[current_character]
```

Instead of:

```
Search the entire right side
```

we do:

```
Constant-time lookup in a 26-element array
```

Therefore:

```
Time Complexity =  $O(n)$ 
Space Complexity =  $O(26)$ 
Space Complexity =  $O(1)$ 
```

20. Optimized $O(n)$ Code

```
def maxIndexDifference(s):
    n = len(s)

    # best[c] stores the farthest ending index
    # reachable from any occurrence of character c
    # processed so far on the right

    best = [-1] * 26

    answer = -1

    # Process from right to left
    for i in range(n - 1, -1, -1):

        current = ord(s[i]) - ord('a')

        # If current character is not 'z',
        # there is a possible next alphabet character
        if current < 25:

            next_char = current + 1
```

```
# If a reachable next character exists
if best[next_char] != -1:

    farthest = best[next_char]

else:

    # No jump possible
    farthest = i

else:

    # 'z' has no next lowercase character
    farthest = i

# Update the best result for current character
best[current] = max(
    best[current],
    farthest
)

# We can start only from 'a'
if current == 0:

    answer = max(
        answer,
        farthest - i
    )

return answer
```

21. Optimized Code: Character Dictionary Version

The array version is fastest and uses constant space.

A dictionary version is sometimes easier to understand:

```
def maxIndexDifference(s):
    best = {}

    answer = -1

    for i in range(len(s) - 1, -1, -1):

        current = s[i]

        if current == 'z':
```

```
        farthest = i

    else:

        next_char = chr(ord(current) + 1)

        if next_char in best:

            farthest = best[next_char]

        else:

            farthest = i

    best[current] = max(
        best.get(current, -1),
        farthest
    )

    if current == 'a':

        answer = max(
            answer,
            farthest - i
        )

    return answer
```

The array version is preferable because lowercase English alphabets have exactly 26 possible characters.

22. Optimized Dry Run on "aaabcb"

Input:

```
s = "aaabcb"
```

Index representation:

```
index:  0  1  2  3  4  5
char:   a  a  a  b  c  b
```

We process from right to left.

i = 5

Current:

```
'b'
```

Required next:

```
'c'
```

There is no processed 'c' on the right.

Therefore:

```
farthest = 5
```

Update:

```
best['b'] = 5
```

Current state:

```
best['b'] = 5
```

i = 4

Current:

```
'c'
```

Required next:

```
'd'
```

There is no 'd' on the right.

Therefore:

```
farthest = 4
```

Update:

```
best['c'] = 4
```

Current state:

```
best['b'] = 5  
best['c'] = 4
```

$i = 3$

Current:

```
'b'
```

Required next:

```
'c'
```

We have:

```
best['c'] = 4
```

Therefore:

```
farthest = 4
```

Update:

```
best['b'] = max(5, 4)  
best['b'] = 5
```

Why do we keep 5?

Because:

```
index 5 ('b') can end at index 5
```

while:

```
index 3 ('b') can end at index 4
```

The better result for character 'b' is:

```
5
```

$i = 2$

Current:

```
'a'
```

Required next:

```
'b'
```

We have:

```
best['b'] = 5
```

Therefore:

```
farthest = 5
```

Distance:

```
5 - 2 = 3
```

Current answer:

```
answer = 3
```

Update:

```
best['a'] = 5
```

$i = 1$

Current:

```
'a'
```

Required next:

```
'b'
```

We still have:

```
best['b'] = 5
```

Therefore:

```
farthest = 5
```

Distance:

```
5 - 1 = 4
```

Update:

```
answer = max(3, 4)
```

```
answer = 4
```

$i = 0$

Current:

'a'

Required next:

'b'

We have:

`best['b'] = 5`

Therefore:

`farthest = 5`

Distance:

`5 - 0 = 5`

Update:

`answer = max(4, 5)`

`answer = 5`

Final answer:

5

23. Complete Optimized Dry Run Table

For:

```
s = "aaabcb"
```

i	Current Character	Required Next	Best Next Result	Farthest Reachable	Distance if 'a'	Answer
5	b	c	none	5	-	-
4	c	d	none	4	-	-
3	b	c	4	4	-	-
2	a	b	5	5	3	3
1	a	b	5	5	4	4
0	a	b	5	5	5	5

Final:

```
Answer = 5
```

24. Why Does the DP Transition Work?

Suppose we are at index **i**.

Let:

```
current = s[i]
```

The next required character is:

```
next = current + 1
```

There are many possible occurrences of **next** to the right.

For example:

```
current index i
```

```
next character occurs at:
```

```
j1
```

```
j2  
j3
```

For each possible jump:

```
i → j1  
i → j2  
i → j3
```

The final result from each destination is already represented by:

```
dp[j1]  
dp[j2]  
dp[j3]
```

Therefore:

```
dp[i] = max(  
    dp[j1],  
    dp[j2],  
    dp[j3]  
)
```

The important optimization is:

We do not need to remember every j .

We only need the best result among all future occurrences of the required character.

So we maintain:

```
best[next_character]
```

Then:

```
dp[i] = best[next_character]
```

If there is no future occurrence of the next character:

```
dp[i] = i
```

because we cannot move.

25. Why Do We Keep the Maximum for Each Character?

Suppose we have processed two occurrences of 'b'.

First occurrence:

```
index 3
```

From index 3, the farthest reachable index is:

```
3
```

So:

```
best['b'] = 3
```

Later, we process another 'b'.

```
index 1
```

From index 1, we can go:

```
1 ('b') → 2 ('c') → 6 ('d')
```

Therefore:

```
farthest = 6
```

Now:

```
best['b'] = max(3, 6)
```

```
best['b'] = 6
```

This means:

Among all processed 'b' states, the best future ending index is 6.

When an earlier 'a' needs a 'b', it can use:

```
best['b'] = 6
```

26. Why Is the Answer Updated Only at 'a'?

The problem says:

We can start only from an index containing 'a'.

Therefore, while processing every index, we calculate its farthest reachable ending index.

But we calculate the final distance only when:

```
s[i] == 'a'
```

Then:

```
distance = farthest - i
```

And:

```
answer = max(answer, distance)
```

If no 'a' exists:

```
answer remains -1
```

Therefore:

```
return -1
```

27. Edge Cases

Case 1: No 'a'

```
s = "xyz"
```

No valid starting index exists.

Answer:

```
-1
```

Case 2: Only 'a'

```
s = "aaaa"
```

There is no 'b'.

Every start immediately stops.

Possible distances:

```
0 - 0 = 0  
1 - 1 = 0  
2 - 2 = 0  
3 - 3 = 0
```

Answer:

```
0
```

Case 3: "ab"

```
s = "ab"
```

Path:

$$0 \text{ ('a')} \rightarrow 1 \text{ ('b')}$$

Distance:

$$1 - 0 = 1$$

Answer:

1

Case 4: "ac"

$$s = \text{"ac"}$$

From 'a', we need 'b'.

There is no 'b'.

Therefore:

$$\text{farthest} = 0$$

Distance:

$$0 - 0 = 0$$

Answer:

0

Case 5: "ba"

$$s = \text{"ba"}$$

The 'a' is at index 1.

There is nothing after it.

Distance:

$$1 - 1 = 0$$

Answer:

0

Case 6: "abcd"

s = "abcd"

Path:

0 → 1 → 2 → 3

Distance:

$$3 - 0 = 3$$

Answer:

3

Case 7: "abcbzzd"

s = "abcbzzd"

Best path:

```
0 → 1 → 2 → 6
```

Distance:

```
6 - 0 = 6
```

Answer:

```
6
```

This example is important because it proves:

```
Always choose the earliest next character
```

is not enough.

And:

```
Always choose the farthest next character
```

is also not enough.

We need the best future result.

28. Why a Simple Greedy Approach Fails

Consider:

```
s = "abcbzzd"
```

```
index:  0  1  2  3  4  5  6
char:   a  b  c  b  z  z  d
```

At index 0:

```
current = 'a'
```

Need:

```
'b'
```

Possible choices:

```
index 1  
index 3
```

If we choose the farthest 'b':

```
0 → 3
```

Now:

```
s[3] = 'b'
```

Need:

```
'c' after index 3
```

There is no 'c'.

So the path stops at:

```
3
```

Distance:

```
3 - 0 = 3
```

But choosing the earlier 'b':

```
0 → 1 → 2 → 6
```

gives:

```
6 - 0 = 6
```

Therefore:

```
Greedy choice does not work.
```

The correct idea is:

```
For each state, use the best possible future state.
```

This is Dynamic Programming.

29. Brute Force to Optimized Transformation

Brute Force

```
for every starting index i:  
    if s[i] == 'a':  
        recursively try every valid j:  
            if j > i  
            and s[j] == next character:  
                dfs(j)
```

The expensive part is:

```
searching every j
```

Memoized DFS

We store:

```
memo[i] = farthest ending index from i
```

This avoids recalculating the same index.

But we still do:

```
for j in range(i + 1, n)
```

for many indices.

Time:

```
O(n2)
```

Right-to-Left DP

Instead of searching all future positions:

```
best[next_character]
```

already contains the best result for that character.

Therefore:

```
current index i  
  
current character = s[i]  
  
next character = current + 1  
  
farthest = best[next character]
```

Then update:

```
best[current character] = max(  
    best[current character],  
    farthest  
)
```

This removes the inner loop.

Final complexity:

$O(n)$

30. The Core $O(n)$ Pattern

The transformation is:

Brute Force:

```
for every i:
    scan all j > i
        if transition(i, j):
            use answer[j]
```

becomes:

Optimized:

scan from right to left:

```
    answer for current state
        = best[next state]

    update best[current state]
```

In this problem:

```
current state = current character

next state = immediate next alphabet character
```

Therefore:

```
best[current character]
```

is enough.

31. General Template

Whenever a problem has:

1. Positions or indices.
2. Movement only to the right.
3. The next state depends only on the current value/character.
4. The number of possible states is small.

Try:

Process from right to left.

Maintain:

`best[state]`

Meaning:

The best future answer available for this state.

Generic form:

```
best = array/dictionary

for i from right to left:
    current_state = state[i]
    next_state = transition(current_state)
    if best[next_state] exists:
        result = best[next_state]
    else:
        result = base_case
    best[current_state] = combine(
        best[current_state],
```

```
    result  
  )
```

32. Pattern Recognition Checklist

When you see a new problem, ask:

Question 1

Can I start from multiple positions?

If yes:

```
Try each possible starting position.
```

Question 2

Do I always move to the right?

If yes:

```
Think about processing from right to left.
```

Question 3

Does the next state depend only on the current state?

Here:

```
'a' → 'b'  
'b' → 'c'  
'c' → 'd'
```

If yes:

```
Store the best result for each state.
```

Question 4

Is the number of possible states small?

Here:

```
26 lowercase letters
```

If yes:

```
Use an array of size 26.
```

Question 5

Does the brute force repeatedly scan future positions?

If yes:

```
Replace the repeated search with:
```

```
best[next_state]
```

33. Important Difference Between Index DP and Character DP

A normal DP might store:

```
dp[i]
```

for every index.

But the optimized solution notices that the transition depends on:

```
character
```

rather than the exact destination index.

Therefore, we can compress:

```
n states
```

into:

```
26 states
```

We do not need:

```
dp[0], dp[1], dp[2], ...
```

in the final optimized solution.

We only need:

```
best['a']  
best['b']  
best['c']  
...  
best['z']
```

This is state compression.

34. Why the Processing Direction Is Right to Left

The current index can only jump to the right.

Therefore, the answer for the current index depends on future indices.

For example:

```
dp[i] depends on dp[j]
```

where:

```
j > i
```

Therefore, when calculating `dp[i]`, we should calculate all larger indices first.

That means:

$$n - 1 \rightarrow n - 2 \rightarrow \dots \rightarrow 0$$

So:

Right-to-left processing

is the natural DP direction.

This is the same general principle as:

If `dp[i]` depends on states to the right, calculate from right to left.

35. Final Optimized Code with Full Comments

```
def maxIndexDifference(s):
    n = len(s)

    # best[c] stores the farthest ending index reachable
    # from any occurrence of character c processed so far.
    #
    # Index mapping:
    #
    # 0 -> 'a'
    # 1 -> 'b'
    # 2 -> 'c'
    # ...
    # 25 -> 'z'

    best = [-1] * 26

    # If there is no 'a', this remains -1.
    answer = -1

    # Process from right to left because
    # every jump goes to the right.
    for i in range(n - 1, -1, -1):

        # Convert current character to a number.
        #
        # 'a' -> 0
        # 'b' -> 1
        # ...
        # 'z' -> 25

        current = ord(s[i]) - ord('a')
```

```
# 'z' has no next lowercase character.
if current == 25:

    farthest = i

else:

    # Immediate next alphabet character.
    next_char = current + 1

    # If a reachable next character exists,
    # jump conceptually to its best result.
    if best[next_char] != -1:

        farthest = best[next_char]

    else:

        # No valid jump exists.
        farthest = i

# Store the best result for the current character.
best[current] = max(
    best[current],
    farthest
)

# Only 'a' can be a starting point.
if current == 0:

    answer = max(
        answer,
        farthest - i
    )

return answer
```

36. Complexity

Let:

```
n = length of the string
```

The string is scanned exactly once:

```
O(n)
```

For every character, we perform only constant-time operations:

```
ord()  
arithmetic  
array lookup  
max()
```

Therefore:

Time Complexity = $O(n)$

The **best** array has size **26**:

Space Complexity = $O(26)$

Since **26** is constant:

Space Complexity = $O(1)$

Final:

Time: $O(n)$
Space: $O(1)$

37. Interview Explanation

A concise interview explanation:

I first model each index as a state. From index **i**, we can move to any later index containing the next alphabet character. A brute-force DFS tries all such transitions, but scanning all future indices causes $O(n^2)$ complexity. Since transitions depend only on the current character and the alphabet has only 26 states, I process the string from right to left. For each character, I maintain the farthest ending index reachable from any occurrence of that character already processed. For the current character, its answer is the stored answer for the next alphabet character, or the current index if no such character exists. This gives $O(n)$ time and $O(1)$ space.

38. One-Minute Revision

Problem:

Start from 'a'.

Jump right.

Next character must be current character + 1.

Maximize:

$\text{ending_index} - \text{starting_index}$

Brute force:

Try every 'a'.

DFS all valid next indices.

Memoized DFS:

$\text{memo}[i] = \text{farthest index reachable from } i.$

Still:

$O(n^2)$

Optimization:

Process right $\rightarrow$ left.

Maintain:

$\text{best}[\text{character}]$

Meaning:

Best/farthest ending index reachable from that character.

Transition:

```
current = s[i]

next = current + 1

if best[next] exists:

    farthest = best[next]

else:

    farthest = i
```

Update:

```
best[current] = max(
    best[current],
    farthest
)
```

If:

```
current == 'a'
```

then:

```
answer = max(
    answer,
    farthest - i
)
```

Complexity:

```
O(n) time

O(1) space
```

39. Final Mental Model

The entire problem can be reduced to:

```
Current character
  ↓
Need immediate next character
  ↓
What is the best result already known
for that next character?
  ↓
Use best[next]
  ↓
Update best[current]
```

For example:

```
At 'a':

    Need 'b'

    result = best['b']

At 'b':

    Need 'c'

    result = best['c']

At 'c':

    Need 'd'

    result = best['d']
```

So the optimized recurrence is:

```
dp[current] =
    best[next_character]
```

If the next character does not exist:

```
dp[current] = current_index
```

Then:

```
best[current] =  
    max(best[current], dp[current])
```

And for every 'a':

```
answer =  
    max(answer, dp[i] - i)
```

That is the complete transition from:

```
Brute Force  
  ↓  
Recursion  
  ↓  
Memoization  
  ↓  
Right-to-Left DP  
  ↓  
Character-State Compression  
  ↓  
O(n) Time and O(1) Space
```